lec12

March 1, 2026

1 Lecture 12: Hypothesis Testing 1 — Likelihood Ratios, Neyman-Pearson

Data 145, Spring 2026: Evidence and Uncertainty

Instructors: Ani Adhikari, William Fithian

Please run all the code cells below before you start reading.

---

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import norm, poisson

plt.style.use('fivethirtyeight')
%matplotlib inline

# Color scheme for comparing test statistics
COLOR_TV = 'steelblue'       # TV distance - students' familiar tool (Data 8)
COLOR_KS = '#785EF0'         # Discrete KS - purple
COLOR_LRT = 'firebrick'      # LRT - the NP-optimal test

# Benford's law: first-digit probabilities
digits = np.arange(1, 10)
benford_probs = np.log10(1 + 1 / digits)
uniform_probs = np.ones(9) / 9
benford_cdf = np.log10(digits + 1)
log_lr_weights = np.log(uniform_probs / benford_probs)

# Significance level for all Benford examples
alpha = 0.01

def compute_stats(sample, n):
    """Compute TV distance, discrete KS, and log-LR for a digit sample."""
    counts = np.bincount(sample, minlength=10)[1:]  # indices 1-9
    proportions = counts / n
```

```
    tvd = 0.5 * np.sum(np.abs(proportions - benford_probs))
    empirical_cdf = np.cumsum(proportions)
    ks = np.max(np.abs(empirical_cdf - benford_cdf))
    log_lr = np.sum(counts * log_lr_weights)
    return tvd, ks, log_lr

def pvalue(observed, null_dist):
    """One-sided p-value: fraction of null distribution >= observed."""
    return np.mean(null_dist >= observed)
```

1.1 1. Hypothesis Testing Framework

1.1.1 Motivating Example: Benford's Law and Fraud Detection

First digits of naturally occurring data (populations, financial figures, physical constants) tend to follow **Benford's law**:

$$\pi_k = \log_{10}\left(1 + \frac{1}{k}\right), \qquad k = 1, 2, \ldots, 9.$$

This gives $\pi_1 \approx 0.301$, $\pi_2 \approx 0.176$, ..., $\pi_9 \approx 0.046$ — a highly nonuniform distribution. It arises naturally when data spans several orders of magnitude.

Let's verify with real data: the populations of US counties.

[2]:
```
# 2020 Census: all 3,221 US county populations
import pandas as pd
county_df = pd.read_csv('us_county_populations_2020.csv')
county_pops = county_df['population'].values

# Extract first digits
first_digits = np.array([int(str(p)[0]) for p in county_pops])

# Observed proportions
obs_counts = np.bincount(first_digits, minlength=10)[1:]  # digits 1-9
obs_props = obs_counts / len(county_pops)

# Example counties for table (spanning several orders of magnitude)
examples = county_df.iloc[[0, 1, 5, 50, 200, 1000, 2500, -1]]
print(f"{len(county_pops):,} US counties (2020 Census)\n")
print(f"{'County':<45} {'Population':>12}  First digit")
print("-" * 67)
for _, row in examples.iterrows():
    name = row['name']
    print(f"{name:<45} {row['population']:>12,}  {str(row['population'])[0]}")

# Bar chart: observed first-digit frequencies vs Benford
fig, ax = plt.subplots(figsize=(8, 4))
```

```
width = 0.35
ax.bar(digits - width/2, obs_props, width, label=f'US counties (n =␣
↪{len(county_pops):,})',
       color='steelblue', alpha=0.8)
ax.bar(digits + width/2, benford_probs, width, label="Benford's law",
       color='gray', alpha=0.6)
ax.set_xlabel('First digit')
ax.set_ylabel('Proportion')
ax.set_xticks(digits)
ax.legend()
ax.set_title("First digits of US county populations (2020 Census)")
plt.tight_layout()
plt.show()
```

```
3,221 US counties (2020 Census)

County                                          Population  First digit
---------------------------------------------------------------------
Los Angeles County, California                  10,014,009  1
Cook County, Illinois                            5,275,541  5
Orange County, California                        3,186,989  3
Marion County, Indiana                             977,203  9
Davis County, Utah                                 362,679  3
Bedford County, Tennessee                           50,237  5
Martin County, Indiana                               9,812  9
Loving County, Texas                                    64  6
```

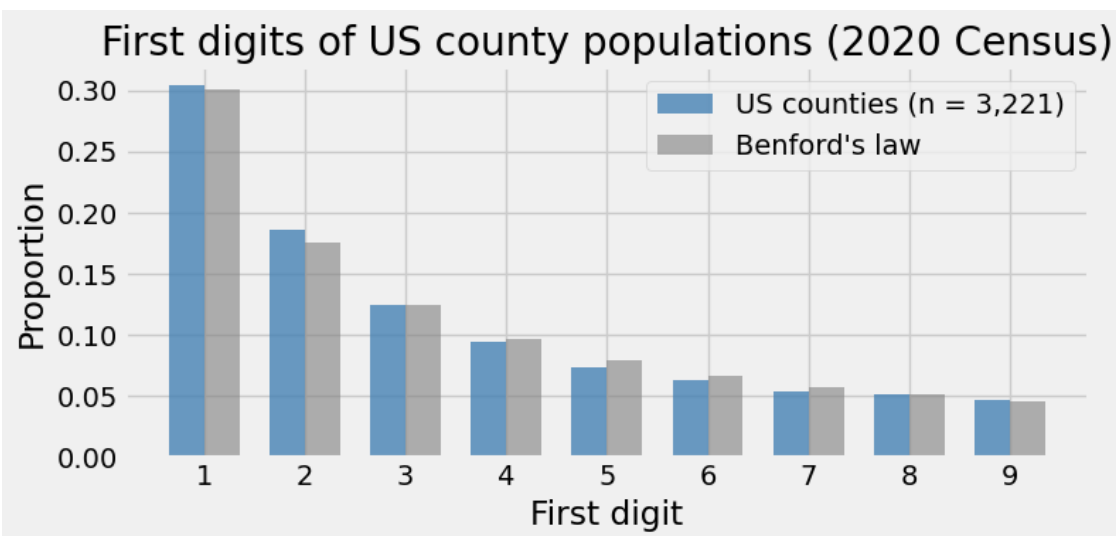


Figure 1. *First digits of 3,221 US county populations (2020 Census) compared with Benford's law. The observed first-digit distribution (blue) closely matches the theoretical Benford probabilities (gray). County populations span five orders of magnitude — from Loving County, TX (population*

64) to Los Angeles County (over 10 million) — exactly the kind of spread that produces Benford's law.

Fraud detection: Fabricated numbers tend to have more uniform first digits — people who invent invoice amounts pick digits 1–9 roughly equally. Given n first digits, we want to test:

$$H_0 : \text{digits follow Benford's law} \qquad \text{vs} \qquad H_1 : \text{digits follow Uniform}\{1, \dots, 9\}$$

You already have **two tools** for this: - **TV distance** (Data 8): $\text{TVD} = \frac{1}{2}\sum_{k=1}^{9} |N_k/n - \pi_k|$ - **Discrete KS statistic** (Lecture 11): $\text{KS} = \max_k |F_n(k) - F_{\text{Benford}}(k)|$

Which is better? Let's find out.

[3]:
```
# Benford vs Uniform first-digit distributions
fig, ax = plt.subplots(figsize=(8, 4))
width = 0.35
ax.bar(digits - width/2, benford_probs, width, label='Benford',␣
 ↪color='steelblue', alpha=0.8)
ax.bar(digits + width/2, uniform_probs, width, label='Uniform',␣
 ↪color='firebrick', alpha=0.8)
ax.set_xlabel('First digit')
ax.set_ylabel('Probability')
ax.set_xticks(digits)
ax.legend()
ax.set_title("Benford's law vs Uniform distribution")
plt.tight_layout()
plt.show()
```

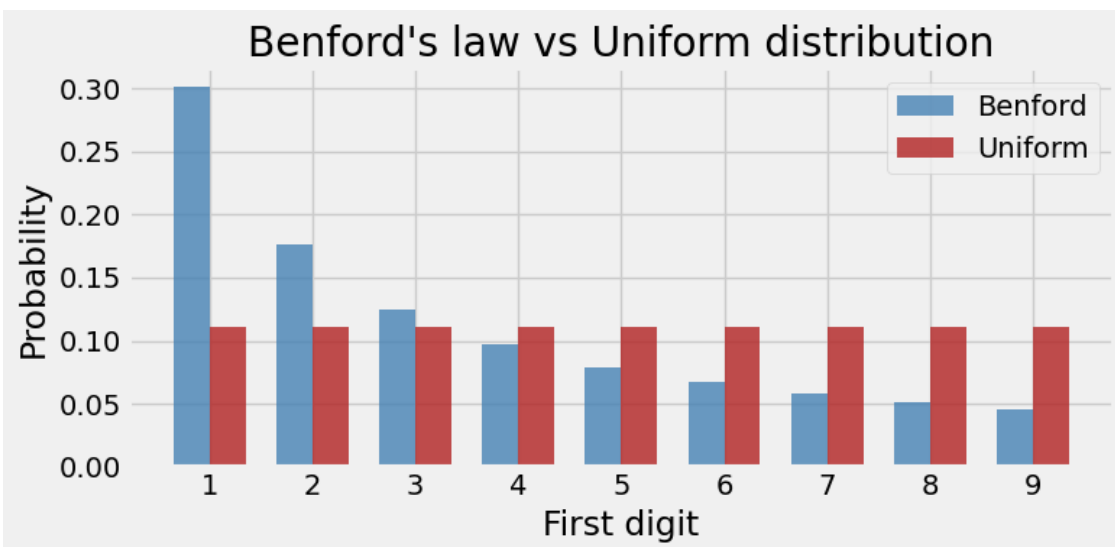


Figure 2. *Benford's law (blue) assigns decreasing probability to each digit: digit 1 has probability* ≈ 0.301*, while digit 9 has only* ≈ 0.046*. The Uniform distribution (red) assigns* $1/9 \approx 0.111$ *to each*

digit. Testing H_0: Benford vs H_1: Uniform amounts to asking whether the first-digit distribution matches the blue bars or the red bars.

[4]:
```
# Pre-compute null and alternative distributions of all three test statistics
# (Run once; used by the live demo and visualization cells below)
n = 25
n_sim = 50_000
rng = np.random.default_rng(42)

# Under H0 (Benford)
null_tvd = np.empty(n_sim)
null_ks = np.empty(n_sim)
null_log_lr = np.empty(n_sim)

for i in range(n_sim):
    sample = rng.choice(digits, size=n, p=benford_probs)
    null_tvd[i], null_ks[i], null_log_lr[i] = compute_stats(sample, n)

# Under H1 (Uniform)
alt_tvd = np.empty(n_sim)
alt_ks = np.empty(n_sim)
alt_log_lr = np.empty(n_sim)

for i in range(n_sim):
    sample = rng.choice(digits, size=n, p=uniform_probs)
    alt_tvd[i], alt_ks[i], alt_log_lr[i] = compute_stats(sample, n)

print(f"Null and alternative distributions simulated (n = {n}, {n_sim:,} reps)")
```

```
Null and alternative distributions simulated (n = 25, 50,000 reps)
```

[5]:
```
# LIVE DEMO: Generate one sample from the Uniform alternative
# Re-run this cell for a fresh draw!
sample = np.random.choice(digits, size=n, p=uniform_probs)
tvd, ks, _ = compute_stats(sample, n)
p_tvd = pvalue(tvd, null_tvd)
p_ks = pvalue(ks, null_ks)

print(f"Sample of n = {n} digits from Uniform:")
print(f"  TV distance = {tvd:.4f}   (p-value = {p_tvd:.4f})")
print(f"  Discrete KS = {ks:.4f}   (p-value = {p_ks:.4f})")
print()
if p_ks < p_tvd:
    print("→ KS gives a smaller p-value (stronger evidence against H )")
elif p_tvd < p_ks:
    print("→ TV gives a smaller p-value this time (but KS usually wins)")
else:
    print("→ Both tests give similar p-values this time")
```

```python
# Where does our observed statistic fall in the null distribution?
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

tvd_bins = np.arange(null_tvd.min() - 0.01, null_tvd.max() + 0.02, 0.02)
ks_bins = np.arange(null_ks.min() - 0.01, null_ks.max() + 0.02, 0.02)

for ax, null_vals, obs_val, p_val, label, color, bins in zip(
    axes,
    [null_tvd, null_ks],
    [tvd, ks],
    [p_tvd, p_ks],
    ['TV Distance', 'Discrete KS'],
    [COLOR_TV, COLOR_KS],
    [tvd_bins, ks_bins]
):
    ax.hist(null_vals, bins=bins, density=True, color=color, alpha=0.5,
            edgecolor='white')
    # Mark observed statistic with brick red arrow
    ylim = ax.get_ylim()
    arrow_y = ylim[1] * 0.85
    ax.annotate(f'observed\n$p = {p_val:.3f}$',
                xy=(obs_val, 0), xytext=(obs_val, arrow_y),
                fontsize=10, color='firebrick', fontweight='bold',
                ha='center', va='bottom',
                arrowprops=dict(arrowstyle='->', color='firebrick', lw=2.5))
    ax.set_title(f'Null distribution: {label}')
    ax.set_xlabel(label)
    ax.set_ylabel('Density')

plt.suptitle(f'Where does this sample fall under $H_0$? (n = {n})',␣
 ↪fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

```
Sample of n = 25 digits from Uniform:
  TV distance = 0.2982   (p-value = 0.0764)
  Discrete KS = 0.2982   (p-value = 0.0065)

→ KS gives a smaller p-value (stronger evidence against H )
```

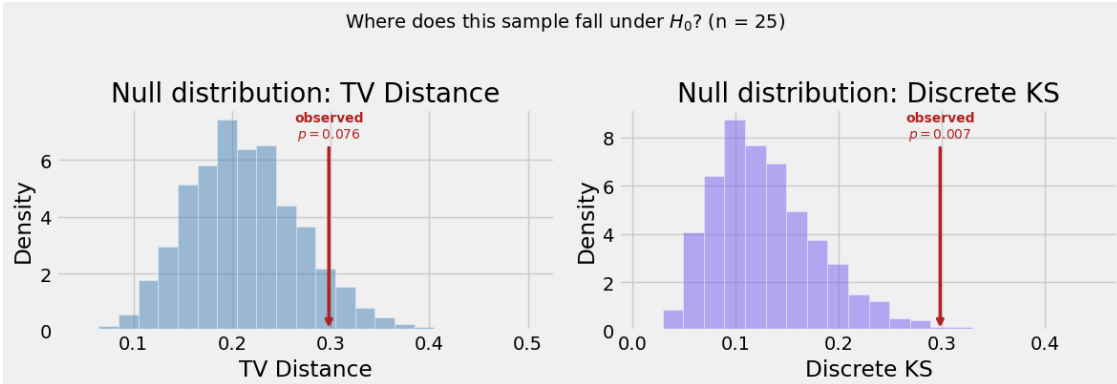


Figure 3. *Null distributions (under Benford's law) of TV distance (left) and discrete KS (right) at $n = 25$. The red arrow marks where this sample's observed statistic falls. Values further to the right are more extreme — more unlikely under H_0.*

[6]:
```
# How would a Uniform sample differ? Null vs alternative distributions
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

tvd_all = np.concatenate([null_tvd, alt_tvd])
ks_all = np.concatenate([null_ks, alt_ks])
tvd_bins = np.arange(tvd_all.min() - 0.01, tvd_all.max() + 0.02, 0.02)
ks_bins = np.arange(ks_all.min() - 0.01, ks_all.max() + 0.02, 0.02)

for ax, null_vals, alt_vals, label, color, bins in zip(
    axes,
    [null_tvd, null_ks],
    [alt_tvd, alt_ks],
    ['TV Distance', 'Discrete KS'],
    [COLOR_TV, COLOR_KS],
    [tvd_bins, ks_bins]
):
    ax.hist(null_vals, bins=bins, density=True, color=color, alpha=0.5,
            edgecolor='white', label='$H_0$ (Benford)')
    ax.hist(alt_vals, bins=bins, density=True, color=color, alpha=1.0,
            histtype='step', linewidth=2, label='$H_1$ (Uniform)')
    ax.set_title(label)
    ax.set_xlabel(label)
    ax.set_ylabel('Density')
    ax.legend(fontsize=9)

plt.suptitle(f'Null vs alternative distributions (n = {n})', fontsize=14, y=1.
 ↪02)
plt.tight_layout()
plt.show()
```

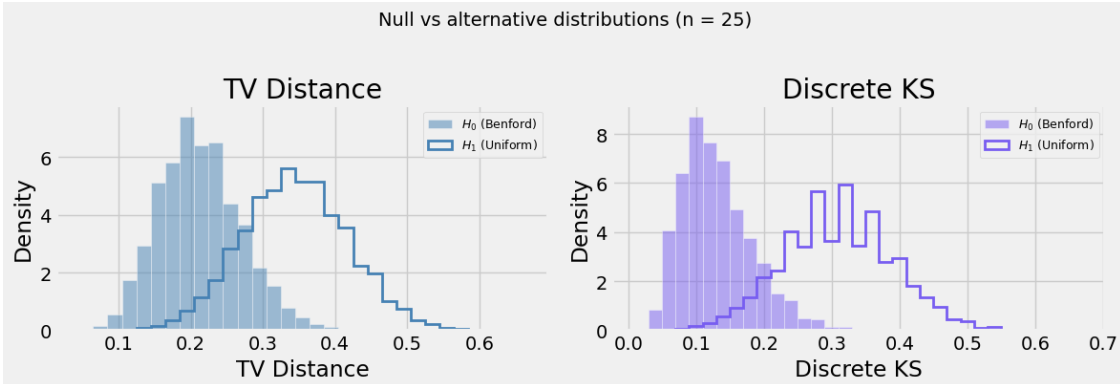


Figure 4. *Null (filled) and alternative (outline) distributions of the TV distance (left) and discrete KS statistic (right) at $n = 25$. Notice the KS distributions are more separated than the TV distributions — that's why KS gets smaller p-values. After we formalize the testing framework, we'll quantify this separation precisely.*

1.1.2 The Question This Motivates

Run the live demo cell a few times. You'll notice that the KS test consistently produces smaller p-values — it's better at detecting the Uniform alternative.

Some tests are better than others. Just like we asked "what makes an estimator good?" in Lectures 3–6, we now ask: **what makes a test good?**

To answer this rigorously, we need to formalize the hypothesis testing framework.

1.1.3 Setup

A **statistical model** $\{P_\theta : \theta \in \Theta\}$ is a family of distributions indexed by a parameter θ.

- **Null hypothesis** $H_0 : \theta \in \Theta_0$ and **alternative hypothesis** $H_1 : \theta \in \Theta_1$, where Θ_0 and Θ_1 are disjoint subsets of Θ (and usually exhaustive: $\Theta_0 \cup \Theta_1 = \Theta$).
- A **simple** hypothesis completely specifies the distribution (e.g., "digits follow Benford"). A **composite** hypothesis doesn't (e.g., $\theta > 0.5$). In the Benford example, H_0 (Benford) is simple, but the alternative could be composite — any non-Benford distribution. For now we focus on the specific simple alternative H_1: Uniform.
- There is a fundamental **asymmetry**: H_0 is the default. We either *reject* H_0 or *fail to reject* it — like a trial where the defendant is innocent until proven guilty.

1.1.4 The Critical Function

A **test** (or *critical function*) is a function $\phi(x) \in [0, 1]$ that maps data x to a rejection probability.

- In practice, $\phi(x) \in \{0, 1\}$ almost always: we either reject or don't. This defines a **rejection region** $R = \{x : \phi(x) = 1\}$.

- Randomized tests ($\phi(x) \in (0, 1)$) are useful in theory for discrete distributions; they are essentially never used in practice.

1.1.5 Error Rates, Level, and Power

- **Type I error** (false positive): rejecting H_0 when it's true
- **Type II error** (false negative): failing to reject H_0 when H_1 is true

The **power function** gives the probability of rejection as a function of θ:

$$\beta_\phi(\theta) = E_\theta[\phi(X)]$$

A test has **level** α if:

$$\sup_{\theta \in \Theta_0} \beta_\phi(\theta) \leq \alpha$$

The goal is to **maximize power at alternative values of** θ — i.e., $\beta_\phi(\theta)$ for $\theta \in \Theta_1$ — while **controlling the Type I error rate at level** α.

The right α depends on context. In Data 8 you probably used $\alpha = 0.05$ — but that's not always appropriate. If we're the IRS deciding whether to audit a tax return, we'd want a much lower threshold: auditing 5% of honest taxpayers would be costly and unfair. We'll use $\alpha = 0.01$ in the Benford example.

1.1.6 Revisiting the Bake-Off with Formal Language

Now we can say it precisely: at $\alpha = 0.01$ and $n = 25$, the KS test has **higher power** than the TV test for detecting Uniform digits vs Benford's law. Let's visualize why — by plotting the null and alternative distributions of each test statistic side by side, with the rejection region marked.

[7]:
```
# Null vs Alternative distributions: TV and KS only - with rejection region
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

tvd_all = np.concatenate([null_tvd, alt_tvd])
ks_all = np.concatenate([null_ks, alt_ks])
tvd_bins = np.arange(tvd_all.min() - 0.01, tvd_all.max() + 0.02, 0.02)
ks_bins = np.arange(ks_all.min() - 0.01, ks_all.max() + 0.02, 0.02)

for ax, null_vals, alt_vals, label, color, bins, stat_name in zip(
    axes,
    [null_tvd, null_ks],
    [alt_tvd, alt_ks],
    ['TV Distance', 'Discrete KS'],
    [COLOR_TV, COLOR_KS],
    [tvd_bins, ks_bins],
    ['TV', 'KS']
):
    ax.hist(null_vals, bins=bins, density=True, color=color, alpha=0.5,
            edgecolor='white', label='$H_0$ (Benford)')
    ax.hist(alt_vals, bins=bins, density=True, color=color, alpha=1.0,
```

```
            histtype='step', linewidth=2, label='$H_1$ (Uniform)')

    # Critical value and rejection region
    crit = np.quantile(null_vals, 1 - alpha)
    beta = np.mean(alt_vals >= crit)
    ax.axvline(crit, color='black', linestyle='--', linewidth=1.5)

    # Shade alternative distribution in rejection region (with partial-bar␣
↪handling)
    alt_counts, alt_edges = np.histogram(alt_vals, bins=bins, density=True)
    for j in range(len(alt_counts)):
        left, right = alt_edges[j], alt_edges[j+1]
        if right <= crit:
            continue  # entirely left of cutoff
        shade_left = max(left, crit)  # partial bar: start at crit
        ax.fill_between([shade_left, right],
                        [alt_counts[j], alt_counts[j]],
                        color=color, alpha=0.25)

    # Power annotation
    ax.text(crit + (bins[-1] - crit) * 0.3, ax.get_ylim()[1] * 0.7,
            f'$\\beta_{{{stat_name}}} = {beta:.2f}$',
            fontsize=12, fontweight='bold', color='black',
            bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.8))

    ax.set_title(label)
    ax.set_xlabel(label)
    ax.set_ylabel('Density')
    ax.legend(fontsize=9)

plt.suptitle(f'Null vs alternative distributions ($\\alpha = {alpha}$, $n =␣
↪{n}$)',
             fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

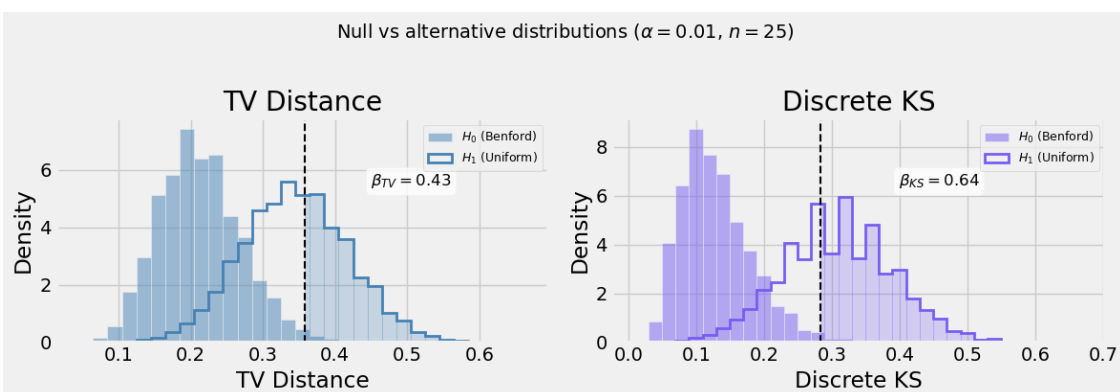


Figure 5. *Null (filled) and alternative (outline) distributions of the TV distance (left) and discrete KS statistic (right) at $n = 25$. The dashed black line marks the rejection threshold at $\alpha = 0.01$. The lightly shaded region shows the alternative distribution's probability mass in the rejection region — that's the* ***power*** *β. The KS test has higher power than the TV test: it captures more of the alternative distribution in the rejection region. Is there a test that does even better? We'll answer this in §3.*

1.2 2. A Sampler of Testing Problems

Before developing theory, let's see the variety of problems that hypothesis testing addresses. Some of these we'll be able to analyze optimally; others require different tools.

1.2.1 Example 1: One-sample z-test (simple hypotheses)

$Z \sim N(\theta, 1)$; test $H_0 : \theta = 0$ vs $H_1 : \theta > 0$. Natural approach: reject for large Z. Under H_0, $Z \sim N(0, 1)$, so we reject if $Z > z_\alpha = \Phi^{-1}(1-\alpha)$. This may seem like a toy example, but the z-test is arguably the **most commonly used test in practice**: any time Z is a standardized summary statistic, we're running a z-test. We'll see specific instances in Lectures 14–15. We'll visualize this test's **power** — the probability of correctly rejecting H_0 — in §4.

1.2.2 Example 2: The t-test (nuisance parameter)

$X_1, \ldots, X_n \overset{\text{iid}}{\sim} N(\mu, \sigma^2)$; test $H_0 : \mu = \mu_0$ vs $H_1 : \mu \neq \mu_0$. If σ^2 were known, this would be a z-test. But σ^2 is unknown — a **nuisance parameter**. Student's t-test replaces σ with S and uses the t-distribution with $n - 1$ degrees of freedom.

1.2.3 Example 3: Comparing two proportions (Fisher exact test)

Two groups: $X \sim \text{Binom}(n_1, p_1)$, $Y \sim \text{Binom}(n_2, p_2)$; test $H_0 : p_1 = p_2$. Under H_0, both proportions equal some unknown common value p — another nuisance parameter. Fisher's insight: **condition on** $X + Y$ to eliminate p. The conditional distribution of X given $X + Y$ is hypergeometric.

1.2.4 Example 4: Two-sample permutation test (nonparametric)

Two samples $X_1, \ldots, X_n$ and $Y_1, \ldots, Y_m$; test $H_0 : F_X = F_Y$. Under H_0, the group labels are exchangeable: every permutation of labels is equally likely. Choose any test statistic; compare to the permutation distribution. You did this in Data 8!

1.2.5 The Big Picture

The z-test is the simplest case; NP theory (next section) will tell us it's optimal. The others involve **nuisance parameters** or **nonparametric nulls** — we'll develop tools for those in Lecture 13.

For now, let's focus on the simplest case: **simple H_0 vs simple H_1**.

1.3 3. The Likelihood Ratio and Neyman-Pearson Lemma

1.3.1 The Likelihood Ratio

Setting: Simple null $H_0 : X \sim P_0$ vs simple alternative $H_1 : X \sim P_1$, where P_0 and P_1 have densities f_0 and f_1 (with respect to some common dominating measure μ).

Define the **likelihood ratio**:

$$\mathrm{LR}(\mathbf{X}) = \frac{f_1(\mathbf{X})}{f_0(\mathbf{X})}$$

The **likelihood ratio test (LRT)** rejects for large $\mathrm{LR}(\mathbf{X})$: data that is much more likely under H_1 than under H_0.

1.3.2 Intuition

Think of building a rejection region one sample point at a time. The power of a test with rejection region R is $\int_R f_1(x)\,d\mu(x)$, while the Type I error is $\int_R f_0(x)\,d\mu(x)$. If the sample space were discrete, both integrals would just be sums over $x \in R$, and we should construct R by collecting sample points for which we can buy the most power $f_1(x)$ per unit of error budget $f_0(x)$ that we must spend.

- **"Bang"** $= f_1(x)$: the power gained by including x in the rejection region
- **"Buck"** $= f_0(x)$: the Type I error cost of including x
- **"Bang for our buck"** $= \mathrm{LR}(x) = f_1(x)/f_0(x)$

Including sample points in order of decreasing LR gives the most power for any given level. This is the **greedy algorithm** for constrained optimization — and the NP lemma says it's optimal.

1.3.3 The Neyman-Pearson Lemma

Statement. Among all level-α tests, the LRT

$$\phi^*(x) = \begin{cases} 1 & \text{if } \mathrm{LR}(x) > c \\ \gamma & \text{if } \mathrm{LR}(x) = c \\ 0 & \text{if } \mathrm{LR}(x) < c \end{cases}$$

where c and γ are chosen so that $E_0[\phi^*(X)] = \alpha$, maximizes power $E_1[\phi^*(X)]$.

Proof (Lagrangian approach). We want to maximize $\int \phi(x) f_1(x)\,d\mu(x)$ subject to $\int \phi(x) f_0(x)\,d\mu(x) \le \alpha$ and $\phi(x) \in [0,1]$.

Form the Lagrangian:

$$\mathcal{L}(\phi;\lambda) = \int \phi(x)\big(f_1(x) - \lambda f_0(x)\big)\,d\mu(x)$$

To maximize over $\phi(x) \in [0,1]$ pointwise: set $\phi(x) = 1$ when $f_1(x) - \lambda f_0(x) > 0$, i.e., when $\mathrm{LR}(x) > \lambda$.

The LRT ϕ^* maximizes this Lagrangian at the multiplier $\lambda = c$. For any competing test ϕ with $E_0[\phi(X)] \le \alpha$:

$$E_1[\phi(X)] \leq E_1[\phi(X)] - c(E_0[\phi(X)] - \alpha) \leq E_1[\phi^*(X)] - c(E_0[\phi^*(X)] - \alpha) = E_1[\phi^*(X)]$$

where the first inequality holds because $E_0[\phi(X)] \leq \alpha$, and the second because ϕ^* maximizes the Lagrangian. □

1.3.4 Randomization at the Boundary

For **discrete** distributions (e.g., binomial), we may not be able to hit exactly level α with a deterministic test. The randomization parameter γ "tops off" the error budget at $\mathrm{LR}(X) = c$.

In practice: just use the conservative (slightly below α) deterministic test. Think of this as "rounding down" ϕ to $\{0, 1\}$, analogous to how we "round up" the p-value to the conservative discrete level.

1.3.5 The NP Lemma in Action: Revisiting Benford

For Benford vs Uniform, the LRT statistic is:

$$\log \mathrm{LR} = \sum_{k=1}^{9} N_k \log \frac{1/9}{\pi_k}$$

This is a **weighted sum of digit counts**. The weights $\log((1/9)/\pi_k)$ are: - **Positive** for digits 5–9 ($\pi_k < 1/9$): seeing these digits is evidence for Uniform - **Negative** for digits 1–4 ($\pi_k > 1/9$): seeing these digits is evidence for Benford

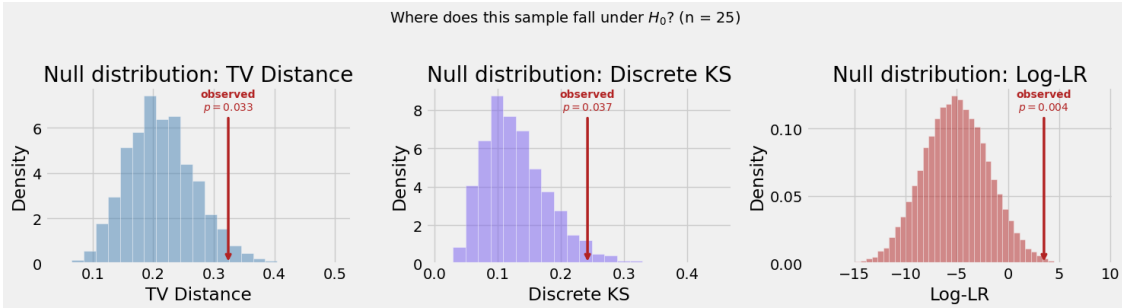
The NP lemma tells us this test must beat both TV distance and KS. Let's verify!

Why might we still use TV or KS in practice? We might not know the alternative! The LRT is optimal for Benford vs Uniform *specifically*, but if the alternative could be any non-Benford distribution, we'd want a more omnibus test. More on this in Lecture 13.

[8]:
```
# LIVE DEMO: Now with all THREE test statistics
# Re-run this cell for a fresh draw!
sample = np.random.choice(digits, size=n, p=uniform_probs)
tvd, ks, log_lr = compute_stats(sample, n)
p_tvd = pvalue(tvd, null_tvd)
p_ks = pvalue(ks, null_ks)
p_lr = pvalue(log_lr, null_log_lr)

print(f"Sample of n = {n} digits from Uniform:")
print(f"  TV distance = {tvd:.4f}   (p-value = {p_tvd:.4f})")
print(f"  Discrete KS = {ks:.4f}   (p-value = {p_ks:.4f})")
print(f"  Log-LR      = {log_lr:.4f}   (p-value = {p_lr:.4f})")
print()
print(f"→ The LRT p-value is {'the smallest!' if p_lr <= min(p_tvd, p_ks) else␣
↪'not the smallest this time (but it usually is).'}")

# Where does our observed statistic fall in each null distribution?
```

```python
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

tvd_bins = np.arange(null_tvd.min() - 0.01, null_tvd.max() + 0.02, 0.02)
ks_bins = np.arange(null_ks.min() - 0.01, null_ks.max() + 0.02, 0.02)
lr_bins = np.linspace(null_log_lr.min() - 0.5, null_log_lr.max() + 0.5, 50)

for ax, null_vals, obs_val, p_val, label, color, bins in zip(
    axes,
    [null_tvd, null_ks, null_log_lr],
    [tvd, ks, log_lr],
    [p_tvd, p_ks, p_lr],
    ['TV Distance', 'Discrete KS', 'Log-LR'],
    [COLOR_TV, COLOR_KS, COLOR_LRT],
    [tvd_bins, ks_bins, lr_bins]
):
    ax.hist(null_vals, bins=bins, density=True, color=color, alpha=0.5,
            edgecolor='white')
    # Mark observed statistic with brick red arrow
    ylim = ax.get_ylim()
    arrow_y = ylim[1] * 0.85
    ax.annotate(f'observed\n$p = {p_val:.3f}$',
                xy=(obs_val, 0), xytext=(obs_val, arrow_y),
                fontsize=10, color='firebrick', fontweight='bold',
                ha='center', va='bottom',
                arrowprops=dict(arrowstyle='->', color='firebrick', lw=2.5))
    ax.set_title(f'Null distribution: {label}')
    ax.set_xlabel(label)
    ax.set_ylabel('Density')

plt.suptitle(f'Where does this sample fall under $H_0$? (n = {n})',␣
 ↪fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

```
Sample of n = 25 digits from Uniform:
  TV distance = 0.3240   (p-value = 0.0330)
  Discrete KS = 0.2421   (p-value = 0.0369)
  Log-LR      = 3.4992   (p-value = 0.0038)

→ The LRT p-value is the smallest!
```



[9]:
```
# Null vs Alternative distributions: ALL THREE test statistics - with rejection␣
↪region
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Shared bins for each statistic
tvd_all = np.concatenate([null_tvd, alt_tvd])
ks_all = np.concatenate([null_ks, alt_ks])
tvd_bins = np.arange(tvd_all.min() - 0.01, tvd_all.max() + 0.02, 0.02)
ks_bins = np.arange(ks_all.min() - 0.01, ks_all.max() + 0.02, 0.02)
lr_bins = np.linspace(
    min(null_log_lr.min(), alt_log_lr.min()) - 0.5,
    max(null_log_lr.max(), alt_log_lr.max()) + 0.5, 50)

for ax, null_vals, alt_vals, label, color, bins, stat_name in zip(
    axes,
    [null_tvd, null_ks, null_log_lr],
    [alt_tvd, alt_ks, alt_log_lr],
    ['TV Distance', 'Discrete KS', 'Log-LR'],
    [COLOR_TV, COLOR_KS, COLOR_LRT],
    [tvd_bins, ks_bins, lr_bins],
    ['TV', 'KS', 'LRT']
):
    ax.hist(null_vals, bins=bins, density=True, color=color, alpha=0.5,
            edgecolor='white', label='$H_0$ (Benford)')
    ax.hist(alt_vals, bins=bins, density=True, color=color, alpha=1.0,
            histtype='step', linewidth=2, label='$H_1$ (Uniform)')

    # Critical value and rejection region
    crit = np.quantile(null_vals, 1 - alpha)
    beta = np.mean(alt_vals >= crit)
    ax.axvline(crit, color='black', linestyle='--', linewidth=1.5)

    # Shade alternative distribution in rejection region (with partial-bar␣
↪handling)
    alt_counts, alt_edges = np.histogram(alt_vals, bins=bins, density=True)
```

```
    for j in range(len(alt_counts)):
        left, right = alt_edges[j], alt_edges[j+1]
        if right <= crit:
            continue  # entirely left of cutoff
        shade_left = max(left, crit)  # partial bar: start at crit
        ax.fill_between([shade_left, right],
                        [alt_counts[j], alt_counts[j]],
                        color=color, alpha=0.25)

    # Power annotation
    ax.text(crit + (bins[-1] - crit) * 0.3, ax.get_ylim()[1] * 0.7,
            f'$\\beta_{{{stat_name}}} = {beta:.2f}$',
            fontsize=11, fontweight='bold', color='black',
            bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.8))

    ax.set_title(label)
    ax.set_xlabel(label)
    ax.set_ylabel('Density')
    ax.legend(fontsize=8)

plt.suptitle(f'Null vs alternative distributions ($\\alpha = {alpha}$, $n =␣
↪{n}$)',
             fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

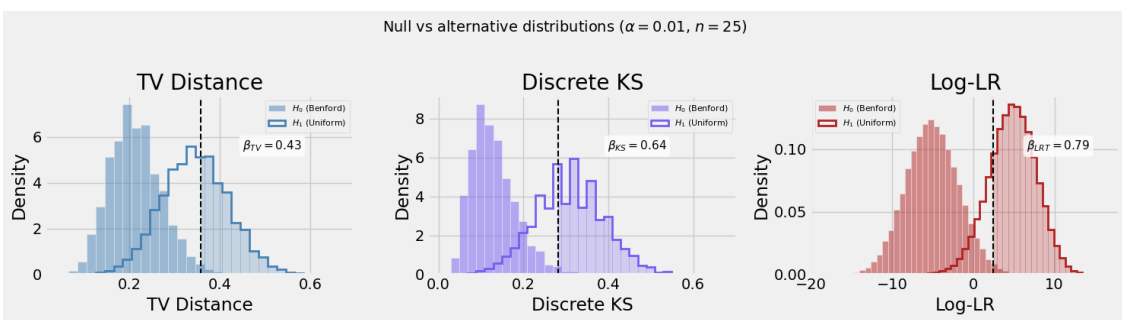
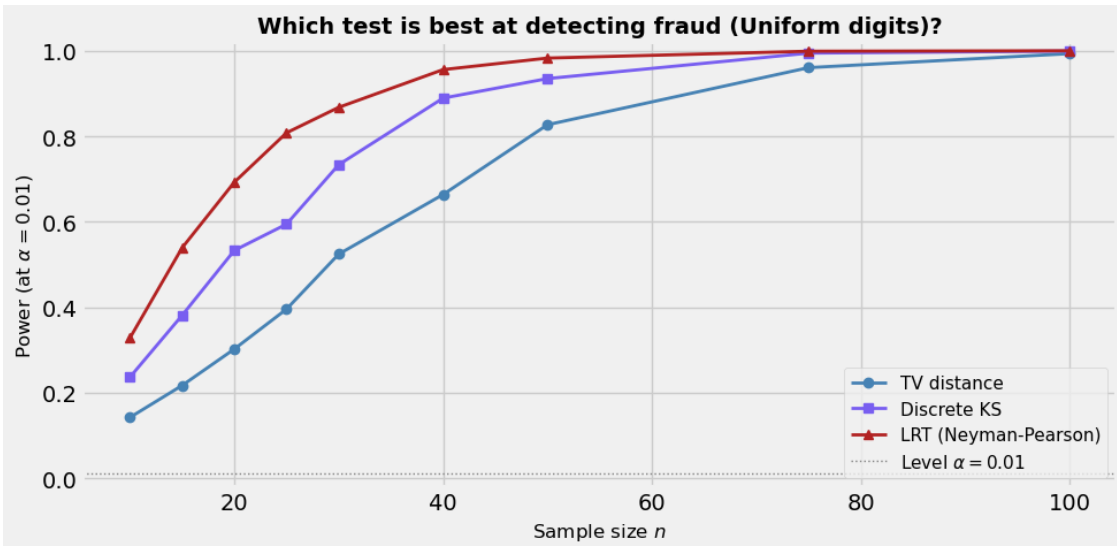


Figure 6. *Null (filled) and alternative (outline) distributions of all three test statistics at $n = 25$, with rejection regions marked. The dashed line is the $\alpha = 0.01$ cutoff; the lightly shaded area is the power β. Compare: the log-LR (firebrick, right) has the largest shaded area — highest power — followed by discrete KS (purple) and TV distance (steelblue). The NP lemma guarantees the LRT is best.*

[10]:
```
# Power curves as a function of n for all three tests
sample_sizes = [10, 15, 20, 25, 30, 40, 50, 75, 100]
n_sim_power = 10_000
rng_power = np.random.default_rng(123)
```

```
powers = {name: [] for name in ['TV', 'KS', 'LRT']}

for n_val in sample_sizes:
    # Simulate null distributions for this n
    null_s = {'tvd': np.empty(n_sim_power), 'ks': np.empty(n_sim_power),
              'log_lr': np.empty(n_sim_power)}
    for i in range(n_sim_power):
        s = rng_power.choice(digits, size=n_val, p=benford_probs)
        null_s['tvd'][i], null_s['ks'][i], null_s['log_lr'][i] =␣
 ↪compute_stats(s, n_val)

    crits = {k: np.quantile(v, 1 - alpha) for k, v in null_s.items()}

    # Simulate under alternative
    rej = {'tvd': 0, 'ks': 0, 'log_lr': 0}
    for i in range(n_sim_power):
        s = rng_power.choice(digits, size=n_val, p=uniform_probs)
        tvd_i, ks_i, lr_i = compute_stats(s, n_val)
        rej['tvd'] += (tvd_i >= crits['tvd'])
        rej['ks'] += (ks_i >= crits['ks'])
        rej['log_lr'] += (lr_i >= crits['log_lr'])

    powers['TV'].append(rej['tvd'] / n_sim_power)
    powers['KS'].append(rej['ks'] / n_sim_power)
    powers['LRT'].append(rej['log_lr'] / n_sim_power)

# Plot
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(sample_sizes, powers['TV'], 'o-', color=COLOR_TV, linewidth=2,␣
 ↪markersize=6,
        label='TV distance')
ax.plot(sample_sizes, powers['KS'], 's-', color=COLOR_KS, linewidth=2,␣
 ↪markersize=6,
        label='Discrete KS')
ax.plot(sample_sizes, powers['LRT'], '^-', color=COLOR_LRT, linewidth=2,␣
 ↪markersize=6,
        label='LRT (Neyman-Pearson)')
ax.axhline(alpha, color='gray', linestyle=':', linewidth=1, label=f'Level␣
 ↪$\\alpha = {alpha}$')
ax.set_xlabel('Sample size $n$', fontsize=12)
ax.set_ylabel(f'Power (at $\\alpha = {alpha}$)', fontsize=12)
ax.set_title('Which test is best at detecting fraud (Uniform digits)?',␣
 ↪fontsize=14,
             fontweight='bold')
ax.legend(loc='lower right', fontsize=11)
```

```
ax.set_ylim(-0.02, 1.02)
plt.tight_layout()
plt.show()

# Print power at n=25
idx = sample_sizes.index(25)
print(f"Power at n = 25, alpha = {alpha}:")
print(f"  TV:  {powers['TV'][idx]:.3f}")
print(f"  KS:  {powers['KS'][idx]:.3f}")
print(f"  LRT: {powers['LRT'][idx]:.3f}")
```




```
Power at n = 25, alpha = 0.01:
  TV:  0.396
  KS:  0.594
  LRT: 0.808
```

Figure 7. *Power of three tests for detecting Uniform first digits vs Benford's law, as a function of sample size n, at $\alpha = 0.01$. The LRT (firebrick triangles) dominates at every sample size, as guaranteed by the Neyman-Pearson lemma. The KS statistic (purple squares) is the next best, followed by TV distance (steelblue circles). The gray dotted line marks the significance level $\alpha = 0.01$.*

1.4 4. Worked Examples

The NP lemma tells us *what* the optimal test is (reject for large LR), but in practice we need to simplify the LR to a "friendly statistic" whose null distribution we can compute. Let's see how this works.

1.4.1 Poisson

Recall the **Poisson distribution**: $X \sim \text{Poisson}(\lambda)$ has PMF

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \qquad k = 0, 1, 2, \ldots$$

Test $H_0 : \lambda = 1$ vs $H_1 : \lambda = 2$. The likelihood ratio is:

$$\text{LR}(X) = \frac{2^X e^{-2}/X!}{1^X e^{-1}/X!} = 2^X \cdot e^{-1}$$

The factor e^{-1} is a constant, so LR is **increasing in** X (since $2 > 1$).

The NP test rejects for large X — this is the same test you'd use intuitively!

Key observation. The rejection region $\{X \geq c\}$ doesn't depend on the specific alternative $\lambda_1 > 1$. We'd get the same test for $\lambda_1 = 1.5$ or $\lambda_1 = 2$ or $\lambda_1 = 10$. This will lead to **uniformly most powerful (UMP) tests** in Lecture 13.

1.4.2 Visualizing Power: The z-test

Now let's visualize the key concept of **power as area**. Consider the z-test from §2: $Z \sim N(\theta, 1)$; test $H_0 : \theta = 0$ vs $H_1 : \theta = 2$.

Draw the null density $N(0, 1)$ and the shifted alternative density $N(2, 1)$ on the same axis. The rejection region $\{Z > z_\alpha\}$ carves out a region in the right tail:

- Area under the **null** density in this region $= \alpha$ (Type I error rate)
- Area under the **alternative** density in this region = **power** $\beta(\theta)$

[11]:
```
# z-test: power as area under null and alternative densities
theta_alt = 2.0
alpha_z = 0.05
z_alpha = norm.ppf(1 - alpha_z)

x = np.linspace(-4, 6, 500)
fig, ax = plt.subplots(figsize=(10, 5))

# Null density
ax.plot(x, norm.pdf(x, 0, 1), color=COLOR_TV, linewidth=2.5, label='$H_0$:␣
↪$N(0, 1)$')
# Alternative density
ax.plot(x, norm.pdf(x, theta_alt, 1), color=COLOR_LRT, linewidth=2.5,
        label=f'$H_1$: $N({theta_alt:.0f}, 1)$')

# Shade rejection region under null (= alpha)
x_reject = np.linspace(z_alpha, 6, 300)
ax.fill_between(x_reject, norm.pdf(x_reject, 0, 1), alpha=0.3, color=COLOR_TV)
# Shade rejection region under alternative (= power)
```

```
ax.fill_between(x_reject, norm.pdf(x_reject, theta_alt, 1), alpha=0.3,␣
 ↪color=COLOR_LRT)

ax.axvline(z_alpha, color='black', linestyle='--', linewidth=1.5,
           label=f'Threshold $z_\\alpha = {z_alpha:.2f}$')

# Annotate
ax.annotate(f'$\\alpha = {alpha_z}$', xy=(z_alpha + 0.3, 0.06), fontsize=12,␣
 ↪color=COLOR_TV)
power_val = 1 - norm.cdf(z_alpha - theta_alt)
ax.annotate(f'$\\beta({theta_alt:.0f}) = {power_val:.2f}$',
            xy=(z_alpha + 0.3, 0.22), fontsize=12, color=COLOR_LRT)

ax.set_xlabel('$Z$', fontsize=12)
ax.set_ylabel('Density', fontsize=12)
ax.set_title(f'Power as area: $z$-test at $\\alpha = {alpha_z}$', fontsize=14,␣
 ↪fontweight='bold')
ax.legend(loc='upper left', fontsize=11)
ax.set_ylim(0, None)
plt.tight_layout()
plt.show()
```

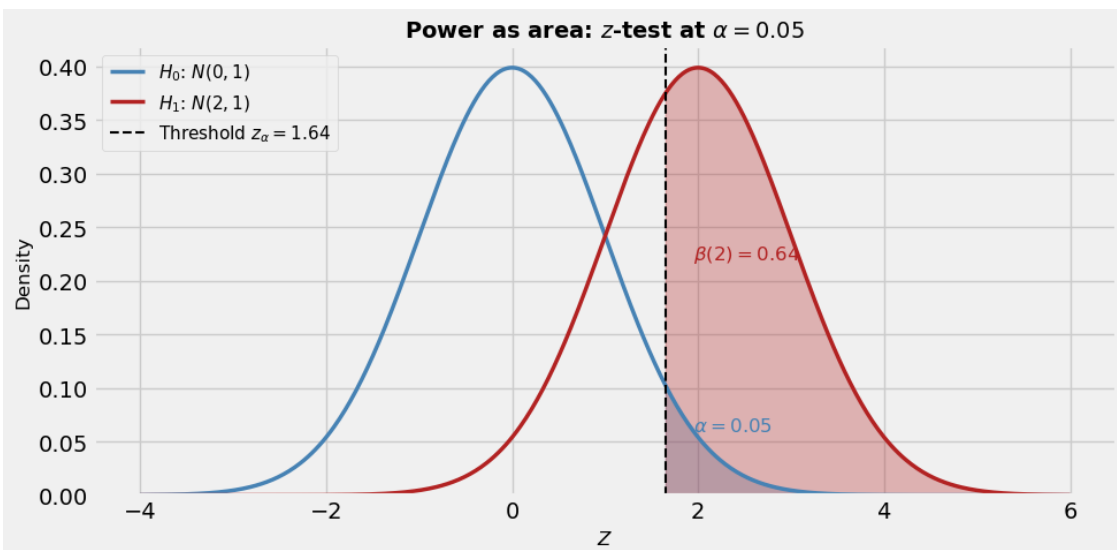


Figure 8. *The z-test for $H_0 : \theta = 0$ vs $H_1 : \theta = 2$. The blue curve is the null density $N(0, 1)$; the red curve is the alternative density $N(2, 1)$. The rejection region $\{Z > z_\alpha\}$ is the right tail beyond the dashed line. The shaded blue area is α (Type I error rate); the shaded red area is $\beta(2)$, the power (probability of correctly rejecting H_0). As θ increases, the red curve shifts further right, putting more area into the rejection region and increasing power.*

1.4.3 Normal Mean (known σ^2)

$X_1, \ldots, X_n \overset{\text{iid}}{\sim} N(\mu, \sigma^2)$; test $H_0 : \mu = \mu_0$ vs $H_1 : \mu = \mu_1 > \mu_0$.

You'll work through the likelihood ratio derivation on Worksheet 6. The key result is that LR is increasing in $\bar{X}$, so the NP test rejects when:

$$\bar{X} \geq \mu_0 + z_\alpha \cdot \frac{\sigma}{\sqrt{n}}$$

Under H_0, $\bar{X} \sim N(\mu_0, \sigma^2/n)$, so this test has exactly level α.

As with the Poisson, the rejection region doesn't depend on the specific μ_1 — only on the direction $\mu_1 > \mu_0$.

1.4.4 Informal p-value

Once we compute $\bar{X}$, the **p-value** is $P_{H_0}(\bar{X} \geq \bar{x}_{\text{obs}})$: how extreme is our observed statistic under the null?

Small p-value $\to$ reject H_0. Specifically, we reject at level α if and only if $p \leq \alpha$.

This is the same quantity `scipy.stats.kstest` computed for us in Lecture 11. We'll give a formal treatment of p-values in Lecture 14.

[12]:
```
# Power function of the z-test for a normal mean
alpha_norm = 0.05
z_alpha_norm = norm.ppf(1 - alpha_norm)
sigma = 1
mu_0 = 0
mu_grid = np.linspace(-1, 4, 500)

fig, ax = plt.subplots(figsize=(10, 5))
for n_val, color in zip([1, 5, 10, 25], ['#648FFF', '#785EF0', '#DC267F',␣
↪'#FE6100']):
    power = 1 - norm.cdf(z_alpha_norm - (mu_grid - mu_0) * np.sqrt(n_val) /␣
↪sigma)
    ax.plot(mu_grid, power, linewidth=2, color=color, label=f'$n = {n_val}$')

ax.axhline(alpha_norm, color='gray', linestyle=':', linewidth=1)
ax.axvline(mu_0, color='gray', linestyle=':', linewidth=1)
ax.set_xlabel(r'True $\mu$', fontsize=12)
ax.set_ylabel(r'Power $\beta(\mu)$', fontsize=12)
ax.set_title(r'Power of the $z$-test ($H_0: \mu = 0$ vs $H_1: \mu > 0$, $\sigma␣
↪= 1$)',
             fontsize=13, fontweight='bold')
ax.legend(fontsize=11)
ax.set_ylim(-0.02, 1.02)
plt.tight_layout()
plt.show()
```

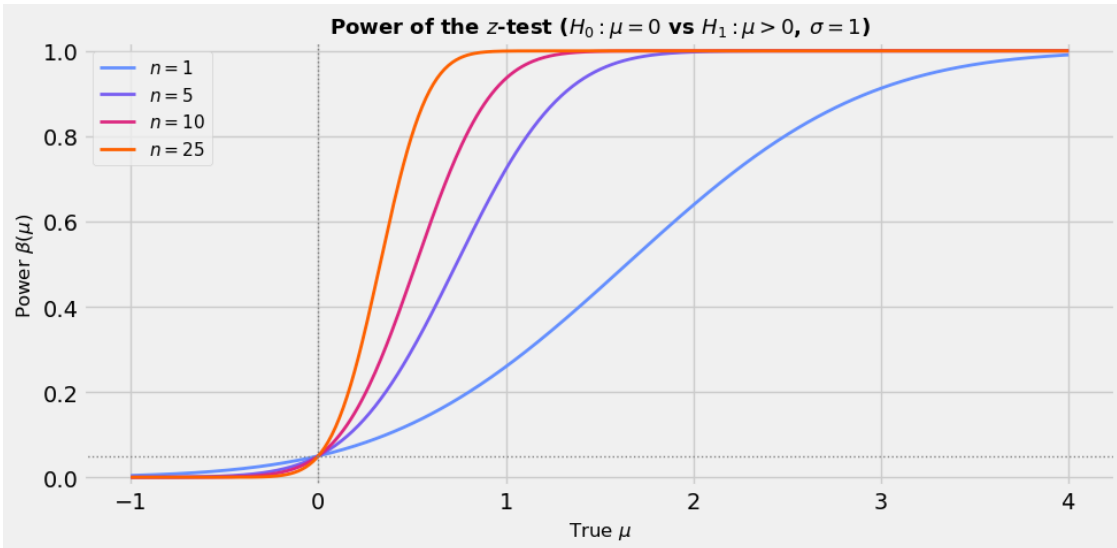


Figure 9. *Power of the one-sided z-test ($H_0 : \mu = 0$ vs $H_1 : \mu > 0$, $\sigma = 1$) at $\alpha = 0.05$, for sample sizes $n = 1, 5, 10, 25$. The power function is $\beta(\mu) = 1 - \Phi(z_\alpha - (\mu - \mu_0)\sqrt{n}/\sigma)$. With just $n = 1$ observation, even $\mu = 2$ gives only moderate power; with $n = 25$, we have near-certain detection for $\mu \geq 1$.*

1.5 5. Summary and Next Time

1.5.1 Today

We started with Benford's law: the KS test beats TV distance, but the **LRT beats both** — the NP lemma tells us why.

Concept	Definition	Intuition
Null/alternative	$H_0 : \theta \in \Theta_0$, $H_1 : \theta \in \Theta_1$ (disjoint)	The null is the default; the alternative is what we're trying to detect
Simple hypothesis	Completely specifies the distribution	e.g., "digits follow Benford" — one distribution, no free parameters
Critical function	$\phi(x) \in [0, 1]$: probability of rejecting H_0 given data x	Usually just a rejection region: reject or don't
Type I error	Rejecting H_0 when it's true (false positive)	Convicting an innocent defendant
Type II error	Failing to reject H_0 when H_1 is true (false negative)	Letting a guilty defendant go free
Power function	$\beta_\phi(\theta) = E_\theta[\phi(X)]$	How likely we are to catch the alternative — this is how we compare tests

Concept	Definition	Intuition
Level α	$\sup_{\theta \in \Theta_0} \beta_\phi(\theta) \leq \alpha$	Our error budget: how much false positive rate we'll tolerate
Likelihood ratio	$\mathrm{LR}(x) = f_1(x)/f_0(x)$	"Bang for buck": power gained per unit of Type I error spent
Neyman-Pearson lemma	The LRT maximizes power among all level-α tests	The greedy algorithm is optimal

1.5.2 Key Takeaways

1. The **NP lemma** says the LRT maximizes power for any given level — it's the best we can do for testing simple vs simple hypotheses
2. The **"bang for buck" intuition**: include sample points in the rejection region in order of decreasing likelihood ratio
3. In practice, we simplify the LRT to a **"friendly statistic"** ($\bar{X}$, X, etc.) and use its null distribution to set the rejection threshold
4. **Hypothesis testing is a rich subject** that extends well beyond simple parametric tests — the t-test, Fisher exact test, and permutation tests handle nuisance parameters and non-parametric settings

1.5.3 Next Time (Lecture 13)

- What if the alternative is composite? **Uniformly most powerful** (UMP) tests
- What if there are nuisance parameters? The t-test, Fisher exact test, and permutation tests as solutions
- Two-sided and multidirectional alternatives: no single test can be best everywhere